

Dialed: reading analog gauges from phone photos with OpenCV 5, and an agent that acts on them

Rohan Borade. Live: localhost:3850. Code: github.com/RohanGlitched/dialed

Summary

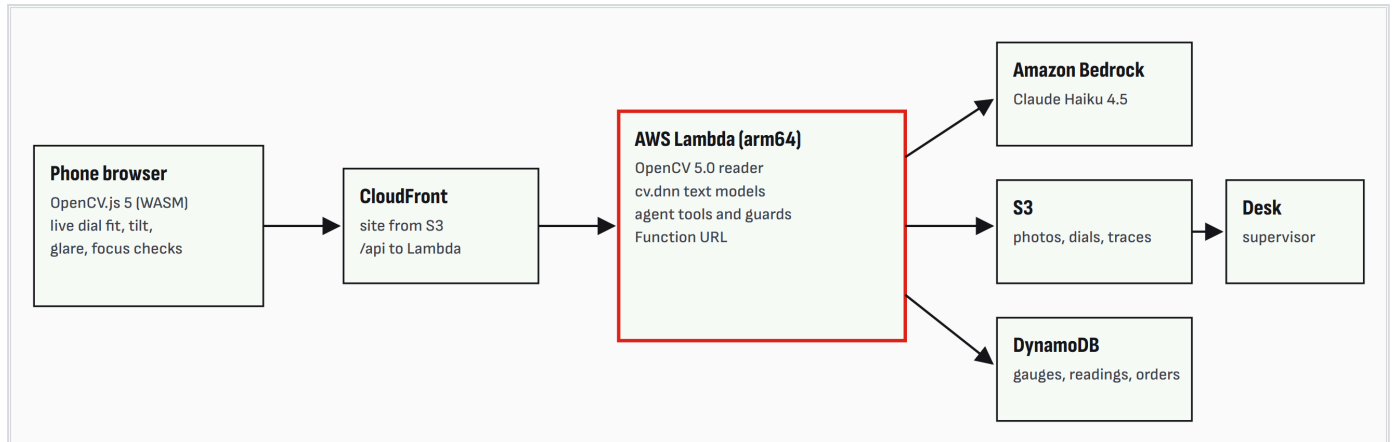
Dialed reads analog pressure and temperature gauges from ordinary phone photos. A classical OpenCV 5 pipeline finds the dial, removes the camera's tilt, locates the true pivot from the tick marks and the needle's own line, reads the printed scale with two OpenCV Model Zoo networks run through `cv.dnn`, and interpolates the needle's value. A calibrated confidence decides whether the reading is trusted. An agent then logs the reading, asks for a better photo with the measured reason, or drafts a maintenance work order that waits for a supervisor. Across 340 test photos, 228 of 231 accepted readings were within 2% of the scale and 1 was off by more than 5%. Real photos are harder than rendered ones: on 37 real gauges used during development, 92% of accepted readings were within 2%, and on the first blind batch, read once before any fix, 4 readings were accepted and 2 of them were within 2%.

1. Problem and users

Most process plants, pump stations and building plant rooms still rely on analog gauges read by people on rounds and written on paper sheets. Readings are copied twice, slow trends go unnoticed until an alarm, missed readings are invisible, and pairs of gauges (a filter's inlet and outlet) are never subtracted. The users are field operators, who need a reading in seconds with a glove on, and maintenance supervisors, who need to trust a reading before they send a fitter.

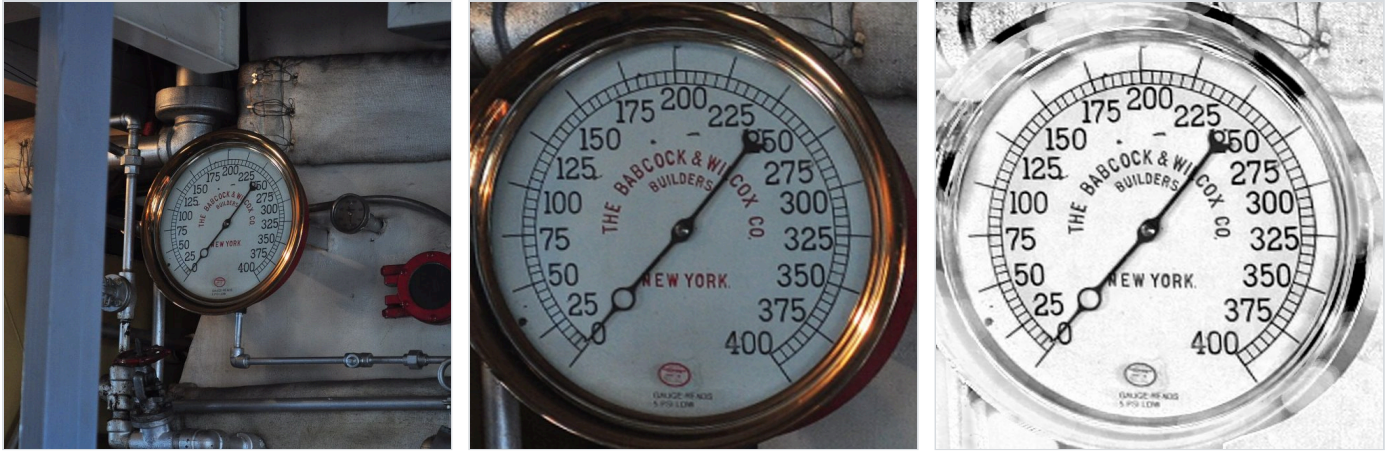
Dialed turns a photo into a trusted reading and does the arithmetic on every reading: the gauge's normal band and alarm limits, the week's drift, and pressure drops between paired gauges.

2. Architecture



OpenCV runs twice: OpenCV.js 5 in the phone's browser coaches the shot; OpenCV 5 on AWS Lambda reads it.

3. The OpenCV 5 pipeline



A real photo (a 1922 steamship's boiler gauge, Joe Mabel, CC BY-SA 4.0), straightened, and its thin-stroke map. Reading: 245 psi, 100% confidence, 36° tilt corrected.

1. **Find the dial.** Canny edges with thresholds from the gradient distribution; ellipses fitted (`fitEllipse`) to edge contours and to Otsu regions, scored by edge support along the outline, size and roundness. A second strong, separate ellipse means two gauges in frame and blocks the read.
2. **Straighten.** One `warpAffine` maps the rim ellipse to a circle (640 px for geometry, 960 px from the full-resolution photo for reading text).
3. **Strokes only.** Morphological black-hat (top-hat on dark faces, chosen by which side the minority pixels fall on) keeps strokes thinner than $\sim 0.11 R$ and drops glare, shading and the face colour.
4. **Tick ring and true centre.** In the polar image (`warpPolar`) the tick band is where high-pass energy along the angle peaks. Each tick is measured as a short line (connected components, principal axis); lines stay straight under perspective, so their least-squares intersection is the pivot, not the ellipse centre.
5. **Needle.** Probabilistic Hough segments (`HoughLinesP`) through the estimated centre are grouped by direction; a hub circle (`HoughCircles`) on the line fixes the pivot. The pointer is the end whose unbroken run from the pivot is longer; counterweight tails are short.
6. **Numbers.** PP-OCRV3 (`TextDetectionModel_DB`) finds text; each box is cut out along its own angle and read by CRNN in upright and inverted orientations (sideways for square boxes), with greedy CTC decoding done in Python to get a per-read confidence.
7. **Scale fit.** Numbers are grouped into rings by radius (two-scale dials), and each ring is fitted by RANSAC over every reading of every number: lost decimal points ("05" is 0.5) and lost minus signs (a value printed twice on a ring) are hypotheses with a cost. The value is interpolated between neighbouring numbers, which absorbs residual perspective error.
8. **Photo checks and confidence.** Tilt from the axis ratio, sharpness (Laplacian variance), glare as compact highlights that wash out the print, distance, needle contrast, fit residual. A logistic model over these measurements, fitted on 600 rendered gauges kept apart from the test sets, gives $P(\text{error} < 2\% \text{ of span})$.

DNN engines. OpenCV 5 offers a new DNN engine beside the classic one. Measured on CPU, the text detector took 357.4 ms on the classic engine and 210.1 ms on the new one, while the recogniser took 43.8 ms per word on the classic engine and 179 ms on the new one. Each model is loaded with the engine that runs it faster.

4. The agent

A tool-calling model (Claude Haiku 4.5 on Amazon Bedrock; NVIDIA Nemotron on Nebius as the fallback; a rule engine if neither answers) chooses among six tools: `read_gauge` (auto, or with the gauge's enrolled scale angles), `compare_history`, `request_reshoot`, `log_reading`, `hold_work_order` and `flag_wrong_gauge`. The vision output changes every later step: a blocking photo issue or low confidence leads to a re-shoot with the measured reason; a unit or range that disagrees with the gauge's registration leads to a tag check; a measured breach leads to a held work order.

Guards in code decide whether each action is allowed: no reading is logged below 90% confidence; a work order needs a measured breach (alarm limit, a week's drift, or the pressure drop across a paired filter) and its priority cannot be set below the breach; every figure the model writes is checked against the measurements and unknown figures are struck through; only a person approves a work order. Each capture's full trace (model turns, tool calls, refusals, timings) is stored with the photo and shown on the approvals desk.

Scenario	Expected	Rule engine	Model
Normal reading	logged	pass	pass
Bearing warning all week	held (drift)	pass	pass
Filter pressure drop over limit	held (differential)	pass	pass
Air receiver over its alarm limit	held (alarm high)	pass	pass
Intake below its alarm limit	held (alarm low)	pass	pass
Shaky photo	reshoot	pass	pass
Photo turned 62° away	reshoot	pass	pass
Wrong gauge photographed	mismatch	pass	pass

5. Deployment on AWS

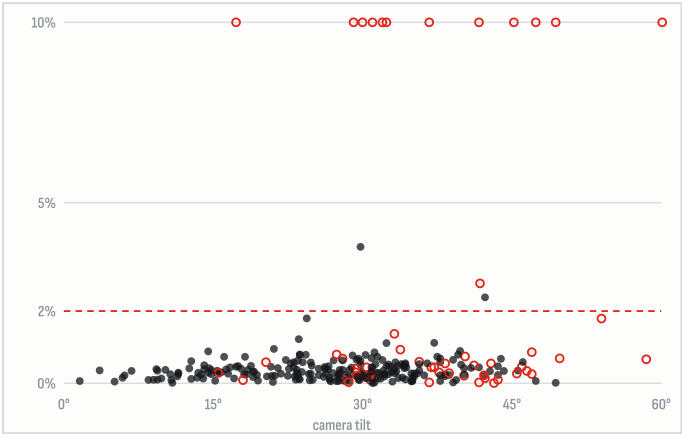
One CloudFormation stack: a Python 3.12 arm64 Lambda function (OpenCV 5.0 pip wheels and both models in the package) behind a Function URL; DynamoDB (gauges, readings, work orders); one S3 bucket for the static site, photos and evidence images; CloudFront in front of all three, with a directory-index function; an error alarm and 14-day logs. `infra/deploy.py` builds Linux wheels without Docker and deploys everything.

6. Evaluation

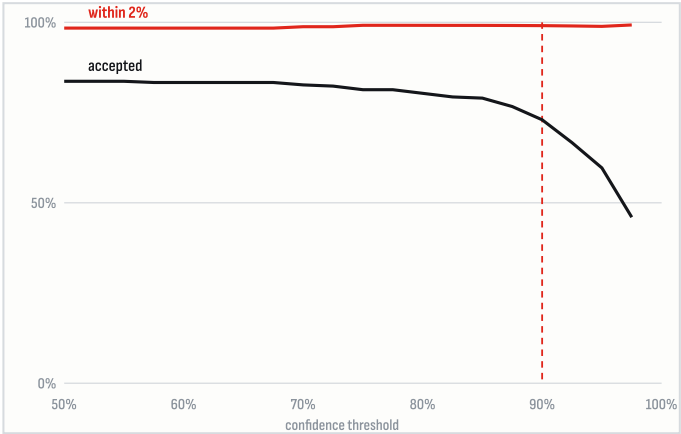
Set	n	Read	Within 2% (all reads)	Accepted	Accepted within 2%	Accepted >5%
Rendered gauges, up to 35° tilt, some glare and blur	200	200	99.0%	178	99.4%	0
Rendered gauges, up to 50° tilt, heavy glare and blur	100	74	81.1%	41	97.6%	0
Real photos from Wikimedia Commons, read by eye (development set)	37	29	65.5%	12	91.7%	1
Real photos from Wikimedia Commons, read by eye, never used for tuning (blind set)	3	1	100.0%	0	0.0%	0

Rendered sets have exact ground truth (the generator places the needle). Real photos are from Wikimedia Commons (CC0, CC BY, CC BY-SA), read by eye from the photo with an uncertainty of about 1% of the scale. They were used during development (failures on them guided the needle and number-reading fixes), so they are a development set and their figures are optimistic; on two-scale dials the reading is scored against the scale the reader used. 46 further real photos that should not be read (two needles, two gauges, the back of a gauge, a dial too small) were accepted 1 times.

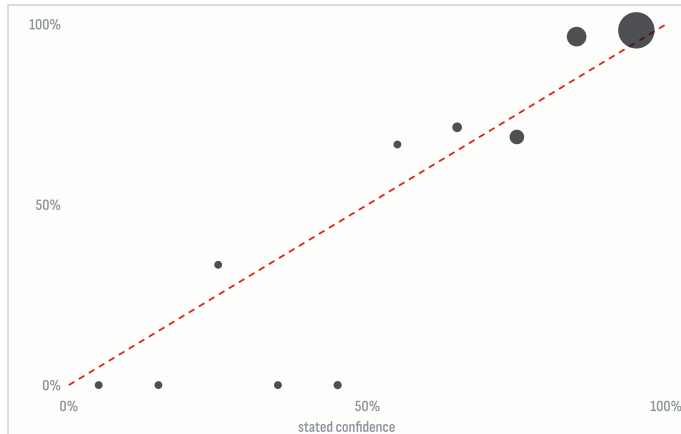
Blind tests. A first batch of 15 real dials and 11 photos to decline was labelled by eye and read once before any change: 4 readings were accepted, 2 within 2% and 1 off by more than 5%, and 1 photo that should have been declined was accepted. Two causes were fixed (print in the blank part of a dial taken for the needle; handwheels and pipe ends taken for a second gauge) and the batch then joined the development set. A second random batch, drawn after the fixes, held only 3 scorable dials among 22 photos: too few for a rate, so they are listed one by one on the Evidence page; 0 of its 19 photos to decline were accepted.



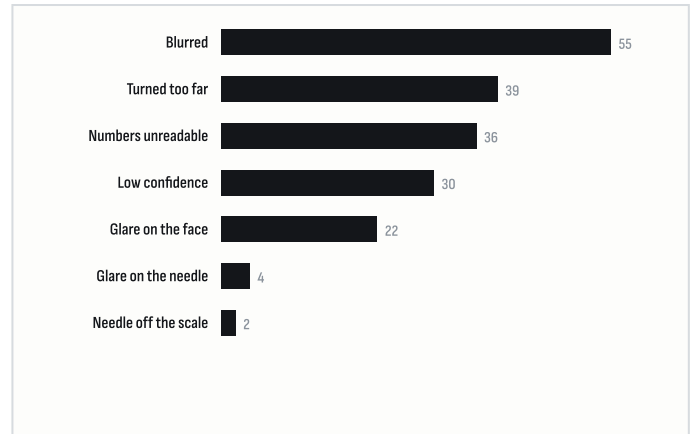
Error against camera tilt (rendered sets). Filled: accepted.



Coverage and accuracy against the threshold.



Stated confidence against observed accuracy.



Why photos were sent back.

7. Limitations

- The text model has no decimal point or minus sign; lost decimals and duplicated negatives are recovered as hypotheses, but vacuum scales labelled on every number can be misread from a single photo (on a round, the gauge's registration catches this).
- Two-needle dials, digital displays and sight glasses are out of scope; small or very dark dials are refused rather than read.
- Ground truth for real photos is a human reading of the same photo.
- A needle resting on its stop below the first printed number (an idle gauge) is extrapolated from the nearest numbers and can read a few percent high; an oven thermometer at room temperature, below its printed scale, was read as a low value instead of being declined.
- Reading the printed numbers dominates the time; a lighter recogniser or caching the scale per enrolled gauge would cut it.

8. Responsible use

No person or face detection is used anywhere; photos are kept only on rounds, as evidence for the reading or work order. The agent cannot act on equipment: it can hold a work order, and only a named person can release it. Uncertainty is always shown: every reading carries its confidence and checks, struck figures stay visible, and refused photos say why. Dialed replaces the clipboard, not the plant's alarms and trips.

9. Reproducibility

scripts/get_models.py (hash-checked models), vision/synth.py (rendered sets), vision/calibrate.py, scripts/evidence.py, scripts/agent_eval.py --model, python -m pytest. Photo credits: CREDITS.md. Code: MIT.